Neural Networks 185 (2025) 107181

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Leveraging neighborhood distance awareness for entity alignment in temporal knowledge graphs

Lin Zhu, Guishun Li, Luyi Bai *

Hebei Key Laboratory of Marine Perception Network and Data Processing, Northeastern University (Qinhuangdao), Qinhuangdao 066004, China

ARTICLE INFO

Keywords:
Entity alignment
Gating mechanism
Graph attention network
Temporal knowledge graph

ABSTRACT

Entity alignment (EA) is a typical strategy for knowledge graph integration, aiming to identify and align different entity pairs representing the same real object from different knowledge graphs. Temporal Knowledge Graph (TKG) extends the static knowledge graph by introducing timestamps. However, since temporal knowledge graphs are constructed based on their own data sources, this usually leads to problems such as missing or redundant entity information in the temporal knowledge graph. Therefore, temporal knowledge graph entity alignment is a meaningful task to avoid the above problems. However, most existing methods tend to ignore the impact of direct and indirect neighborhood information on EA. Therefore, we propose a temporal knowledge graph entity alignment method with neighborhood distance awareness, namely Tem-DA. Tem-DA models direct and indirect neighbors separately, capturing directly connected neighbors with related entities and indirectly connected neighbors with unrelated entities through a distance detection module. Furthermore, we propose a gating mechanism weight the elements in the embedding matrix and through cross-entropy loss function with regularization terms to address adaptive fusion. This mechanism dynamically adjusts the weights of different feature embeddings, thus providing a more flexible and adaptive feature fusion strategy compared with traditional linear weight adjustment methods. Tem-DA effectively captures the temporal information of entities and take advantage of overlapping properties that may have multiple identical time intervals between different entities to encode the temporal information. For some entities that may lack sufficient temporal information, Tem-DA estimates the temporal characteristics of adjacent information and generates temporal embedding vectors. Experimental results on two monolingual TKG datasets show that Tem-DA outperforms popular baseline methods.

1. Introduction

Knowledge graphs (KG) usually represent real-world concepts and their interrelationships in the form of triples (h, r, t) to support downstream tasks such as recommender systems (He et al., 2019), question answering systems (He et al., 2015) and knowledge representation systems (Gai et al., 2016). KG also contains temporal information and is usually stored as part of the attributes or relationships of entities. In contrast, Temporal Knowledge Graph (TKG) will more clearly regard temporal information as a core feature and extend the triple form to the quadruple form (h, r, t, τ) with timestamp, such as YAGO3 (Biega et al., 2013), Wikidata (Krötzsch et al., 2014) and ICEWS (Lautenschlager & Ward, 2015). These studies (Zhen Han et al., 2020) (Liu et al., 2022) (Ding et al., 2023) made notable contributions to explainable forecasting for temporal knowledge graphs.In Tresp et al. (2017),he knowledge graph is extended to the time event of a quadruple. (Zhen Han et al., 2020) roposed a Graph Hawkes Neural Network for temporal knowledge graph. Since existing TKGs are independently constructed based on their respective data sources, this often causes problems such as accurate information or incomplete knowledge coverage. Therefore, in order to provide richer data for downstream applications, the TKG entity alignment task is crucial. Most of the current TKG entity alignment methods are proposed by combining the KG entity alignment method with time encoding.

The key idea of static knowledge graph alignment methods is to represent entities and relationships in knowledge graphs as low-dimensional vectors, which are mainly divided into methods using translation and methods using GNN. Methods using translation, such as TransE (Garcia-Duran et al., 2013), which interpret relationships as translations from head entities to tail entities, have shown good

* Corresponding author.
E-mail address: blovdew@hotmail.com (L. Bai).

https://doi.org/10.1016/j.neunet.2025.107181
Received 29 April 2024; Received in revised form 13 January 2025; Accepted 14 January 2025
Available online 15 January 2025
0893-6080/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

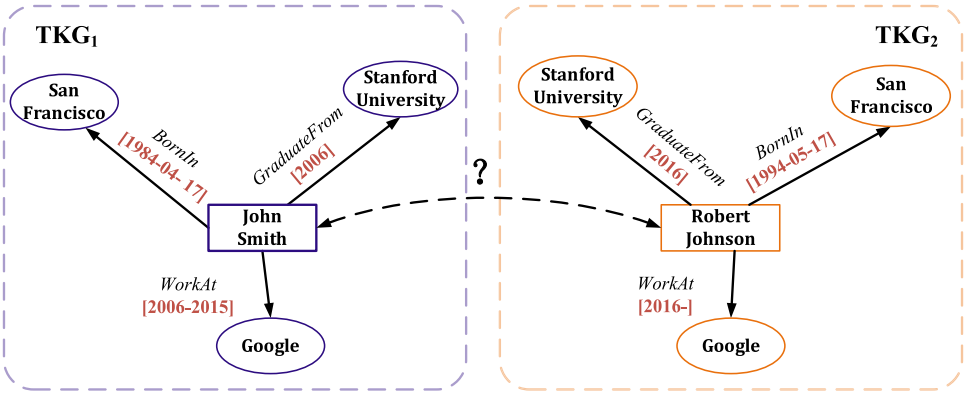

Fig. 1. An example of temporal knowledge graph entity alignment.

performance in KG completion. It is further improved by many methods, such as TransH (Feng et al., 2014), TransR (Sun et al., 2015) and TransD (Ji et al., 2015). MTransE (Yang et al., 2017) performs cross-language entity alignment by spatially transforming the monolingual KG embedding space. Others will use attribute information to complete entity alignment, such as JAPE (Hu et al., 2017) by using relationship and attribute embedding to refine the KG embedding for entity alignment. In recent years, people have accomplished the task of entity alignment by using methods based on graph neural networks (GNN), especially graph convolutional networks (GCN), to effectively capture the neighborhood information of entities. For example, GCN-Align (Lan et al., 2018) processes attribute information as part of the relation triples, and does not learn to represent the attribute triples independently. GMNN (Yu et al., 2019) formulates entity alignment as graph matching between two knowledge graphs. The above GCN-based method gives the same weight to the neighbor information of the entity during embedding, ignoring the different importance of each neighborhood. Therefore, the graph attention network (GAT), as its extension, dynamically adjusts the weight of neighborhood information through the attention mechanism. For example, KECG (Li et al., 2019) combines the GAT model and the TransE model to solve the problem of structural differences between knowledge graphs. In addition to the structural information learned from the structure of relational triples, MRAEA (Mao et al., 2020) enhances the performance of entity alignment by introducing the concept of meta-relationships. Although the above entity alignment methods for static knowledge graphs have made significant progress, they do not fully take into account the particularity of TKGs, that is, the existence of temporal information in TKGs. This may lead to incorrect predictions of entities in relational triples that have similar neighborhood structures but in different temporal contexts. As shown in Fig. 1, the entity "John Smith" and the entity "Robert Johnson" may have different alignment results in the static knowledge graph and the temporal knowledge graph. If in a static knowledge graph, these two entities can be considered to be aligned, but in a temporal knowledge graph, the result is exactly the opposite. In fact, these two entities cannot be considered the same. Therefore, time information plays a very important role in TKG entity alignment.

Current TKG entity alignment research methods often integrate time encoding information into static KG entity alignment methods in order to obtain more accurate entity alignment results. For example, TEA-GNN (Su et al., 2021) and TREA (Xiong et al., 2022) incorporate temporal information and timestamps together into the GNN architecture. Furthermore, Tem-EA (Song et al., 2022) independently encodes temporal information and combines the GCN model to learn structural information embedding. These TKG research methods have verified the importance of time information in entity alignment. However, due to pattern heterogeneity, the direct neighbors of corresponding entities are often dissimilar, thus generating excessive noise.

Therefore, this paper proposes a new method Tem-DA that is suitable for temporal knowledge graph entity alignment. Tem-DA uses GAT to model direct neighbor and indirect neighbor information respectively, thereby capturing the impact of direct neighbors and indirect neighbors on the entity alignment task. In addition, Tem-DA utilizes a gating mechanism to solve the adaptive fusion of different feature embeddings. This mechanism dynamically adjusts the weights of different feature embeddings, thereby providing a more flexible and adaptive feature fusion strategy compared with the traditional linear weight adjustment method. Tem-DA introduces a temporal encoder module, whose purpose is to construct temporal encoding based on the overlapping characteristics of time intervals between entities. For entities that lack sufficient temporal information, Tem-DA generates temporal embedding vectors by estimating the temporal characteristics of adjacent information.

- We propose an effective entity alignment method, Tem-DA, which models the direct and indirect neighbor information of entities respectively, and integrates a gating mechanism.
- To better capture the dynamics in temporal knowledge graphs, Tem-DA exploits the overlapping characteristics of entity time intervals to encode temporal information.
- To optimize the alignment quality and make up for the shortage of pre-aligned entity pairs, Tem-DA proposes a mechanism that combines dynamic distance threshold adjustment with a bootstrap iteration strategy.

2. Related work

2.1. Static knowledge graph entity alignment

Most existing static knowledge graph entity alignment methods are mainly divided into two types: using translation and using graph neural networks. The translation methods treat relations in knowledge graphs as "translations" in a vector space, linking head and tail entities. Models such as TransE and its variants learn representations for entities and relations. MTransE (Yang et al., 2017) embeds entity and relation predicates from various KGs into different embedding spaces, while IPTransE (Liu et al., 2017) and BootEA (Zhang et al., 2018) use an iterative strategy to augment the seed alignment set with newly aligned entities. Other methods, such as JAPE (Hu et al., 2017) and AttrE (Trisedya et al., 2019), use attribute triples to enhance entity alignment, while KDCoE (Chen et al., 2018) treats textual descriptions of entities in KG as special attribute triples. However, these models have limited performance due to the variability of attribute triplets. Recent approaches employ GNN-based approaches to improve entity alignment performance, exploiting the inherent graph structure of knowledge graphs. GCN-Align (Lan et al., 2018) and GMNN (Yu et al., 2019) learn representations using attribute triples as relation triples, while MuGNN (Cao et al., 2019) uses a multi-channel GNN to encode KG across multiple channels. To address structural heterogeneity, NMN (Wu et al.,

2020) learns KG structural information and neighborhood differences. RMHN (J. Chen et al., 2024) designs a new relational mapping based on high-order graph convolutional networks for entity alignment, focusing on high-order neighbors. HGCN (Y. Wu et al., 2019) combines highway network structure and graph convolutional network to achieve adaptive information flow control. Similarly, RDGCN (Y. Wu et al., 2019) utilizes entity graphs and relationship graphs to encode adjacent structures and relationships across entities. To emphasize the scalability of the entity itself, HOLI-GNN (J. Chen et al., 2024) uses the local expansion mechanism to amplify the characteristics of entities, which helps to better retain important information in the graph structure. However, these methods are not applicable to TKG. To address this problem, we introduce a new approach, namely Tem-DA.

2.2. Temporal knowledge graph entity alignment

In recent years, there have been relatively few studies on entity alignment in temporal knowledge graphs, and most of them were research on time embedding methods in temporal knowledge graphs. Han et al. (Zhen Han et al., 2021) conclude that compared with time-dependent entity embeddings, timestamp embeddings can achieve the same or even better performance with significantly fewer parameters in terms of time information utilization. DE-SimplE (Goel et al., 2020) employs diachronic feature embedding functions in static KG embeddings to represent entities at distinct timestamps, while retaining the same scoring function as SimplE (Kazemi & Poole, 2018). Know-Evolve (Trivedi et al., 2017) focuses on entities that evolve nonlinearly over time and models the occurrence of facts as a point-in-time process. AtiSE (Xu et al., 2019) decomposes additional time series data to incorporate temporal information into both entity and relational representations. DyERNIE (Zhen Han et al., 2021) allows entity representations to evolve according to velocity vectors defined in the tangent space at each timestamp. ConT (Ma et al., 2019) generates a new core tensor for each timestamp, allowing for temporal data integration. TA-DistMult (Dumančić et al., 2018) and TA-TransE (Dumančić et al., 2018) use recurrent neural networks (RNN) to dynamically capture the importance of entities at different times, strengthen the model's understanding of how relationships change over time, and use DistMult (Yang et al., 2015) and TA-TransE (Dumančić et al., 2018) respectively. The scoring function in TransE is evaluated. In recent years, some research on TKG entity alignment has emerged. However, these research methods usually add time encoding information to the static KG entity alignment research method to generate more accurate entity alignment results. For example, TEA-GNN (Su et al., 2021) and TREA (Xiong et al., 2022) incorporate temporal information with timestamps into the GNN structure. TuckER (Zhen Han et al., 2020) proposes a new continuum model that extends the idea of neural ordinary differential equations to multi-relation graph convolutional networks to deal with continuous temporal changes. In addition, Tem-EA (Song et al., 2022) encodes temporal information alone and learns structural information embedding combined with GCN model. CTEA (Bai et al., 2024) uses time as part of a triples for representation learning and proposes a novel similarity method. In order to handle the correlation between relationships, TKGA-RP (Wang et al., 2024) first constructs forward and reverse inverse relationships with the beginning and end of time. A new method is then proposed to capture the relational association (RA) of forward and reverse relationships by considering interacting entities. The TKG research approach described above demonstrates the role of temporal information in entity alignment. However, these research methods do not consider the extent to which indirect neighborhood information contributes to entity alignment.

Table 1
Commonly used symbols and terms.

Symbols and terms	Descriptions
$(\mathscr{E},\mathscr{R},\mathscr{A},\mathscr{T},\mathscr{V},\mathscr{R}_{\mathscr{T}},\mathscr{A}_{\mathscr{T}})$	A temporal knowledge graph
$\mathscr{E}$	A set of entities
$\mathscr{R}$	A set of relation predicates
$\mathscr{A}$	A set of attribute predicates
$\mathscr{T}$	A set of timestamps
$\mathscr{V}$	A set of attribute values
$\mathscr{R}_{\mathscr{T}}$	A set of relational triples
$\mathscr{A}_{\mathscr{T}}$	A set of attribute triples
z_t	Update gate
r_t	Reset gate
$\widetilde{h}_t$	Candidate hidden state
h_t	Final hidden state

3. Methodology

3.1. Problem formulation

Notation In order to be as consistent as possible with the notation and terminology in existing papers, we have developed standard notation and terminology conventions in this paper. And the common symbols and terms are summarized are shown in Table 1.

TKG The representation form of the temporal knowledge graph can be defined as $\mathscr{TKG} = (\mathscr{E},\mathscr{R},\mathscr{A},\mathscr{T},\mathscr{V},\mathscr{R}_{\mathscr{T}},\mathscr{A}_{\mathscr{T}})$. Among them, $\mathscr{R}_{\mathscr{T}} = (\mathscr{E},\mathscr{R},\mathscr{E})$ is the set of relation triples, and $\mathscr{A}_{\mathscr{T}} = (\mathscr{E},\mathscr{A},\mathscr{V})$ is the set of attribute triples.

Entity alignment of TKGs Entity alignment uses the pre-aligned entity alignment seed set S to identify and determine a set of uniquely matching entity pairs $\{(e_i,e_j), e_i \in \mathscr{E}_1, e_j \in \mathscr{E}_2\}$ between two temporal knowledge graphs$\mathscr{TKG}_1 = (\mathscr{E}_1, \mathscr{R}_1, \mathscr{A}_1, \mathscr{T}_1, \mathscr{V}_1, \mathscr{R}_{\mathscr{T}1}, \mathscr{A}_{\mathscr{T}1})and\mathscr{TKG}_2 = (\mathscr{E}_2,\mathscr{R}_2,\mathscr{A}_2,\mathscr{T}_2,\mathscr{V}_2,\mathscr{R}_{\mathscr{T}2},\mathscr{A}_{\mathscr{T}2})$ of the same language, and they represent the real-world same object.

3.2. Overview

As shown in Fig. 2, Tem-DA contains two modules: entity embedding module and entity alignment module. Among them, the entity embedding module is further subdivided into three parts: structural information embedding, temporal information embedding and attribute information embedding.

In the structural information embedding, the distance detection module is first used to distinguish the distance information of the entity's direct neighbors and indirect neighbors. The GAT model then embeds the neighbor information at different distances separately to generate structural feature embeddings for each entity. Throughout the embedding process, the embedding output of each layer is controlled through a gating mechanism to achieve effective integration of different features. For temporal information, a temporal encoder is introduced to construct a temporal encoding based on the overlapping characteristics of time intervals between entities. The attribute information is embedded by the BERT model to obtain the feature vector of the attribute information.

In the entity alignment module, joint embedding learning is used to fuse structure embedding, temporal embedding and attribute embedding to generate target entity embeddings. In order to combine features from these different sources, the adaptive adjustment of different feature embeddings is completed through the gated recurrent unit GRU, providing a more flexible and adaptive feature fusion scheme than the traditional linear weight mediation method. In order to improve the accuracy of entity alignment in temporal knowledge graphs, this paper proposes a mechanism that combines dynamic distance threshold adjustment and self-service iteration strategies. Next, each of the above modules will be introduced in turn.

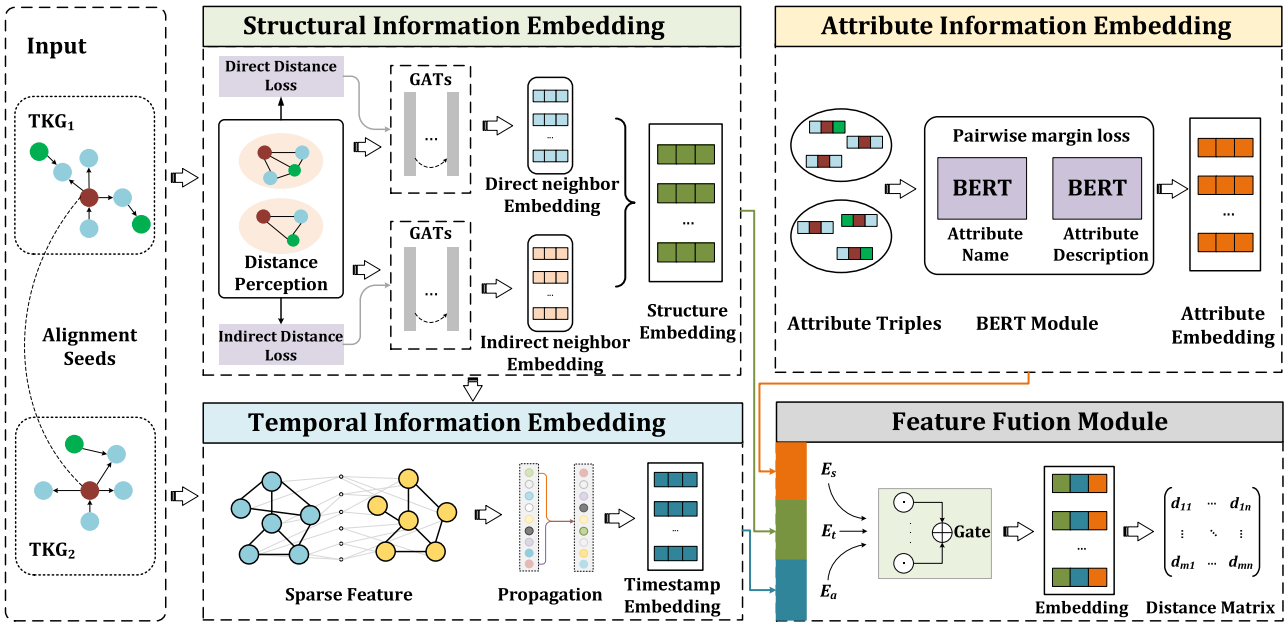

Fig. 2. The brief framework of Tem-DA.

3.3. Structural information embedding

To solve the impact of direct neighbors and indirect neighbors of equivalent entities in different knowledge graphs on entity embedding, we use the GAT model to perform embedding learning on the direct neighbors and indirect neighbors of entities respectively. The specific content is introduced as follows:

3.3.1. Distance perception module

In order to solve the impact of neighbors with different intervals in entity alignment, we adopt a distance detection module to distinguish the distance between entity neighbors in the temporal knowledge graph. This is because distance differences between entity neighbors have a significant impact on entity alignment results.

The core of the distance detection module is a two-layer fully connected feed-forward neural network. Its function is to process the quadruple data of the TKG and classify the distance between entity neighbors. The quadruple structure (head entity, relationship, tail entity, timestamp) of the temporal knowledge graph provides rich contextual information, and the distance detection module learns this information to distinguish direct and indirect connections between entity neighbors.

This module first preprocesses the quadruple data to adapt to the network's data requirements, and then inputs it into the distance detection module to determine the distance information. Among them, the entity category exists in the form of *one-hot* encoding as a distance type for classifying entity neighbors.

The distance detection information output by the distance detection module can represent the relative distance of the entity's neighbors in the embedding space and realize three classifications of interconnections. This classification is based on the direct connection, indirect connection or no connection status between entities. To optimize the distance detection module, we define a loss function and train the module by comparing the predicted distance classification results with the actual *one-hot* encoded labels and minimizing the loss. The loss function is shown in Eq. (1):

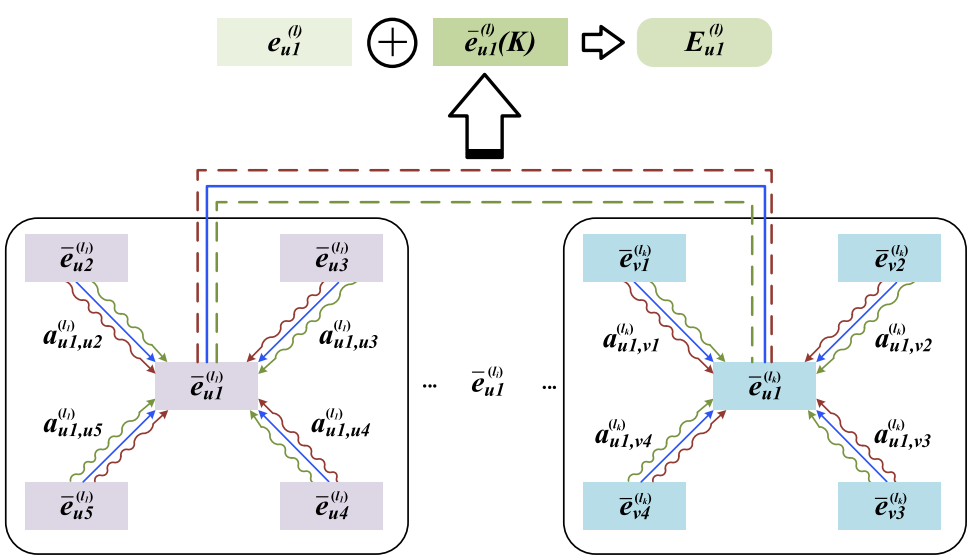

Fig. 3. GAT embedding diagram. E_{u1} indicates the structural embedding information of entity e_{u1} combined with neighbor information.

$$L_D = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{3} y_{ic}\log p_{ic} \tag{1}$$

where N is the total number of entities in TKG; c represents the index of the category. There are three possible categories, which is in line with the description of the above module dividing the relationships between entities into three types; y_{ic} represents the true value of the label, if the i_{th} If the sample belongs to category c, it is 1, otherwise it is 0; p_{ic} is the prediction of sample i in category c, this is obtained with the output information of the distance detection module.

3.3.2. Embedding based on gating mechanism

In the field of entity alignment for TKG, graph convolutional network (GCN) has become a commonly used proximity embedding strategy. However, GCN-based methods have a limitation, which is to give each neighboring node the same weight when calculating entity embedding, resulting in the target entity embedding failing to distinguish the difference in influence of neighbor information. In actual scenarios, the importance of the influence of different neighbors on entities is often different.

Based on this understanding, we propose to use the graph attention network (GAT) to encode the entity structure information in TKG in this paper. The GAT embedding is shown in Fig. 3. GAT assigns different weights to each neighbor node through an adaptive attention mechanism, thereby more precisely capturing the different degrees of influence of neighbor nodes on the target entity. The purpose of this processing method is to distinguish neighbor information that is more important for entity embedding and provide powerful clues for more efficient entity alignment. In view of the specific characteristics of the neighbor structure characteristics and distance information of entities in TKG, we design an embedding initializer, which is composed of two layers of GAT. This double-layer GAT can distinguish between direct and indirect neighbors of entities, thereby effectively capturing the structural differences between entities, and constructing a primary embedding representation space accordingly. Tem-DA integrates a gating mechanism into the GAT model, aiming to enhance the flexibility of the model when processing different levels of information fusion. This mechanism is dedicated to dynamically adjusting the information flow between hidden layers to optimize the performance of the overall network structure. Through such a gating mechanism, the model can adaptively filter and integrate embedded information from different levels, ensuring the effective transmission of information in deep networks. The attention coefficient and node feature update formulas in the GAT layer are shown in Eq. (2) and Eq. (3):

$$a_{ij}^{(l)} = \frac{\exp\left(LeakyReLU\left(p^{(l)}\left[W^{(l)}h_i^{(l)} \| W^{(l)}h_j^{(l)}\right]\right)\right)}{\sum_{k\in N_i}\exp\left(LeakyReLU\left(p^{(l)}\left[W^{(l)}h_i^{(l)} \| W^{(l)}Rh_k^{(l)}\right]\right)\right)} \tag{2}$$

$$\overline{e}_i^{(l)} = \sigma\left(\sum_{j\in N_i} a_{ij}^{(l)} W^{(l)} e_j^{(l)}\right) \tag{3}$$

where $h_i^{(l)}$ and $h_j^{(l)}$ represent the hidden states of node i and node j in the L-th layer respectively; $a_{ij}^{(l)}$ represents the attention coefficient between node i and node j in the L-th layer; $\overline{e}_i^{(l)}$ represents the feature representation of node i in the L-th layer; σ refers to the activation function $ReLU$; the attention parameter vector $p^{(l)}$ of the lth layer maps the high-dimensional feature vector into a real number; $W^{(l)}$ is a learnable weight matrix used to combine the original features of the node mapped to embedding space.; $\|$ represents the connection operation; $LeakyReLU$ is the nonlinear activation function.

In order to enable the GAT model to understand graph data more comprehensively, it provides a more refined and comprehensive node representation than a single attention mechanism. This model introduces a multi-head attention mechanism to increase the stability of the model. As shown in Eq. (4), the graph attention layer is run K times in parallel, using a different weight matrix W each time, and then the resulting feature vectors are merged, which is the final node representation.

$$\overline{e}_i^{(l)}(K) = \|_{k=1}^{K}\sigma\left(\sum_{j\in N_i}\left[a_{ij}^{(l)}\right]^k\left[W^{(l)}\right]^k e_j^{(l)}\right) \tag{4}$$

where $\left[a_{ij}^{(l)}\right]^k$ represents the attention coefficient of node i on the k attention heads of the L-th layer; $\left[W^{(l)}\right]^k$ represents the k attention heads of the L-th layer. trainable weight matrix on.

Inspired by Sun et al. (Dai et al., 2020) uses gating mechanisms to control and regulate information flow in graph neural networks. It determines the transmission and filtering of information by introducing gating units. In our proposed model Tem-DA, by embedding gated recurrent units (GRU) between GAT layers, we can more finely control and optimize the process of information transmission and filtering. GRU mainly contains two gates: update gate and reset gate. The update gate is responsible for deciding how much information from the previous state needs to be retained into the current state, while the reset gate manages the extent to which existing information is ignored. Such a mechanism means that if the value of the update gate is large, the current state will retain more historical information; if the value of the reset gate is small, then more past state information will be ignored. In the multi-layer GAT structure, the introduction of GRU allows us to use its gating mechanism to dynamically adjust the flow and fusion of information between consecutive layers. The output of each layer of GAT is not only the input of the next layer, but also becomes the input of GRU. Through the gating mechanism of GRU, GAT can retain important features while transferring information between layers, or discard unnecessary information. The GRU related equations is shown as follows:

$$z_t = \sigma(W_z\cdot[h_{t-1}, x_t] + b_z) \tag{5}$$

$$r_t = \sigma(W_r\cdot[h_{t-1}, x_t] + b_r) \tag{6}$$

where W_z and W_r are weight matrices; and b_z and b_r are reset items.

Next, the GRU will calculate candidate hidden states to determine how the unit updates important components of its active or hidden state. Candidate hidden states allow the network to capture long-term and short-term dependencies in sequence data. The calculation process of candidate hidden states is shown in Eq. (7):

$$\widetilde{h}_t = tanh(W\cdot[r_t * h_{t-1}, x_t] + b) \tag{7}$$

Finally, the candidate hidden state $\widetilde{h}_t$ is combined with the output of the update gate z_t to calculate the final hidden state. The calculation process is shown in Eq. (8):

$$h_t = z_t * h_{t-1} + (1 - z_t) * \widetilde{h}_t \tag{8}$$

This equation shows how the update gate controls how much of the old state h_{t-1} is retained and how much new state $\widetilde{h}_t$ is added to the final hidden state h_t. If the update gatez_t is close to 1, then more old states tend to be retained; if the update gate z_t is close to 0, then the updated new states tend to be retained. This mechanism enables GRU to flexibly maintain or update its internal memory state while handling temporal dependencies in sequence data.

3.4. Temporal information embedding

Given that the two entities corresponding to the two temporal knowledge graphs to be aligned record the same set of time intervals and store information in the form of quadruples, we can reasonably infer that if the two corresponding entities in the two temporal knowledge graphs

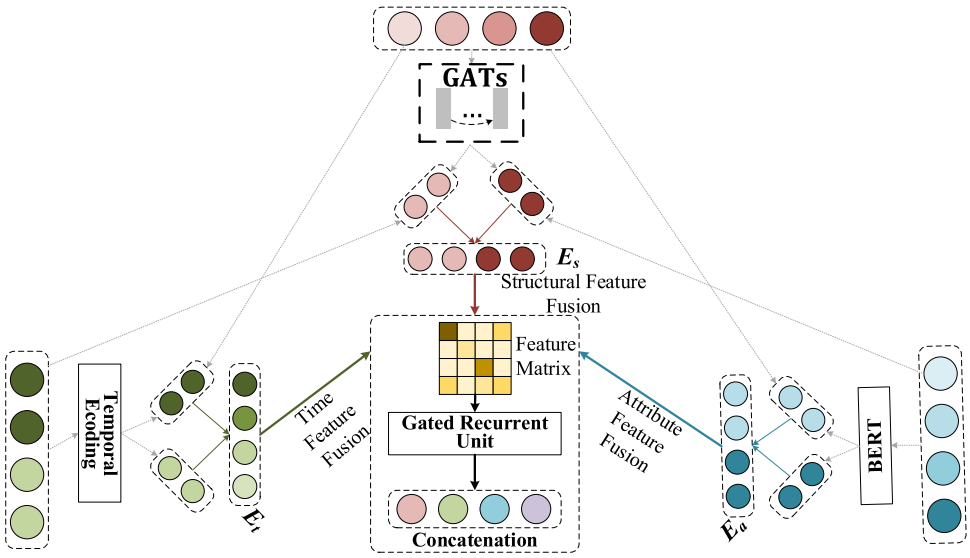

Fig. 4. Feature fusion module based on gating mechanism.

Entities $e_1 \in E_1$ and $e_2 \in E_2$ have multiple coincident points in the time intervals involved in their associated quadruples, then this pair of entities e_1 and e_2 is likely to represent the same object in the real world.

Therefore, Tem-DA proposes a temporal encoder to construct temporal encodings for entities in temporal knowledge graphs based on the characteristics of their time intervals overlapping. In this method, we first extract the bipartite graph B_{et} between entities and timestamps. For each $b_{et} \in B_{et}$ in the graph, Eq. (9) defines the encoding weight as:

$$b_{et} = \frac{\exp(|Q_{et}|)}{\sum_{t \in T} \exp(|Q_{et}|)} \tag{9}$$

where Q_{et} represents the number of four-tuple sets associated with entity e at time t; and $|Q_{et}|$ represents the size of the set, that is, the number of four-tuples involving e and t. This encoding method makes the weight proportional to the relevance of the entity at a specific point in time, thereby effectively utilizing temporal information to promote accurate alignment of entities between maps.

Using the temporal feature matrix as a bridge, corresponding to B_t^{g1} and B_t^{g2} of the temporal knowledge graph $\mathscr{TKG}_1$ and $\mathscr{TKG}_2$ respectively, we can form the alignment matrix S by calculating the pairwise similarity of these two matrices. The alignment process can be expressed as matrix multiplication and transposition operations, and the calculation process is shown in Eq. (10):

$$S = B_t^{g1} \left(B_t^{g2}\right)^T \tag{10}$$

Although we have adopted the temporal feature matrix to guide the EA of the $\mathscr{TKG}_s$, we noticed that when the temporal information of some entities is insufficient, it may lead to unsatisfactory alignment effects. To address this difficulty, Tem-DA proposes a strategy to overcome the lack of temporal information by aggregating the neighbor information of entities. Assuming that sufficient temporal information exists between neighbors, this information can help enhance the alignment quality. On this basis, we adopt a multi-layer graph convolution network according to Wang et al. (2021), and obtain an aggregated feature matrix H^t with rich features through forward propagation of the L layer. The expression is as follows: $H^t = \left[B_t || BB_t || B^2B_t || \ldots || B^LB_t\right]$, where L is the number of layers in the graph convolution network, an adjustable hyperparameter; $\|$ represents the matrix connection operation along the last dimension; B^l Represents the adjacency matrix with distance l in the $\mathscr{TKG}$.

To further consider the impact of different relationship types in the TKG, Tem-DA constructs a relational feature matrix (Xie et al., 2022). For each element $b_{ij} \in B$ in the matrix, the calculation formula is as shown in Eq. (11):

$$b_{ij} = \frac{\sum_{r_j \in R_{(i,j)}} \ln\left(\sum_{r_j \in R_{(i,j)}} \ln\left(|Q| \big/ \left|Q_{r_j}\right|\right)\right)}{\sum_{k \in N_i} \sum_{r_k \in R_{(i,k)}} \ln\left(\sum_{r_j \in R_{(i,j)}} \ln\left(|Q| \big/ \left|Q_{r_j}\right|\right)\right)} \tag{11}$$

where N_i represents the neighbor set of entity e_i, $R_{(i,j)}$ is the relationship set between e_i and e_j, $|Q|$ and $|Q_r|$ represent all four-tuples and the total number of four-tuples containing relationship r respectively.

3.5. Attribute information embedding

At present, research on entity alignment in knowledge graphs has made significant progress in processing attribute triples. For example, AttrE (Trisedya et al., 2019) proposed by Trisedya and JAPE (Hu et al., 2017) proposed by Sun can effectively enhance the performance of EA through attribute triples. The BERT model, because of its excellent language pre-training capabilities, has become a popular choice for embedding attribute information. The attribute triplet of entity e is input into the BERT model, and the CLS embedding is processed through the multi-layer perceptron MLP layer to obtain the feature vector representation of the attribute information, as shown in Eq. (12).

$$M(e) = MLP(CLS(e)) \tag{12}$$

The BERT model uses the margin-based loss function shown in Eq. (13) to optimize the model output:

$$L = \sum_{(e_i, e_j) \in S} \sum_{\left(e_i, e_j'\right) \in S'} max\left\{0, d(e_i, e_j) - d\left(e_i, e_j'\right) + \gamma\right\} \tag{13}$$

where $(e_i, e_j) \in S$ is pre-aligned entites; $\left(e_i, e_j'\right) \in S'$ is negative entity pairs, which replace the rightly aligned counterpart of e_i with e_j' randomly sampled from E_2. $d(e_i, e_j)$ is L_1 distance between attribute information embedding $M(e_i)$ and $M(e_j)$ obtained through BERT embedding. γ is a margin regulating positive pairs and negative pairs.

3.6. Feature fusion module

In order to achieve effective fusion of structure, time and attribute embedding matrices, the Tem-DA method proposes a feature fusion method based on a gating mechanism. This method can not only

Algorithm 1
Entity alignment optimization module.

Input: $\mathcal{TKG}_1 = (\mathcal{E}_1, \mathcal{R}_1, \mathcal{A}_1, \mathcal{T}_1, \mathcal{V}_1, \mathcal{R}_{\mathcal{T}1}, \mathcal{A}_{\mathcal{T}1})$ and $\mathcal{TKG}_2 = (\mathcal{E}_2, \mathcal{R}_2, \mathcal{A}_2, \mathcal{T}_2, \mathcal{V}_2, \mathcal{R}_{\mathcal{T}2}, \mathcal{A}_{\mathcal{T}2})$, Dataset $D = \{(e_i, e_j), e_i \in \mathcal{TKG}_1, e_j \in \mathcal{TKG}_2\}$,seed alignment S, threshold θ_{init}, θ_{final}, θ, maximum iteration count T .
Output: Set of aligned entity pairs P.

```
Initialize θ ← θ_init
for t = 1 to T do
    Train the model using S with current θ
    for each pair (e_i, e_j) in S do
        if dist(e_i, e_j) < θ then
            Add (e_i, e_j) to P
        end if
    end for
    Update θ based on the performance of P on S
    if convergence is detected based on S then
        θ_final ← θ
        break
    end if
end for
for each (e_i, e_j) in D do
    Calculate the distance dist(e_i, e_j) between the entity pair (e_i, e_j) vectors by Equation (3.17)
    if dist(e_i, e_j) < θ_final then
        P←P ∪ {(e_i, e_j)} and S ← S ∪ {(e_i, e_j)}
    end if
end for
return P
```

effectively integrate and improve the complementarity of various types of embedding information, but also adaptively determine the relative importance of different embeddings in entity alignment. As shown in Fig. 4, in the Tem-DA framework, GAT is used to obtain the structure embedding matrix E_s, the time embedding matrix E_t is constructed by analyzing the time correlation between entities, and the BERT model is used to embed attribute information to obtain the embedding matrix E_a. To integrate this embedded information, here are the steps Tem-DA takes:

First, the structure embedding E_s, time embedding E_t and attribute embedding E_a are merged to form a comprehensive feature matrix. Then, input the comprehensive feature matrix into the gate control unit, and the calculation formula is as shown in Eq. (14):

$$w = \sigma\left(W_g * [E_s; E_t; E_a] + b_g\right) \tag{14}$$

where W_g is the weight matrix; b_g is the bias vector; σ represents the sigmoid function.

Finally, the gating weight w is used to weight the elements in the embedding matrix element-by-element, and is added to obtain the fused entity embedding. The calculation formula is shown in Eq. (15):

$$E = w_1 * E_s + w_2 * E_t + w_3 * E_a \tag{15}$$

In the training phase of EA, we also use the cross-entropy loss function with regularization terms to adjust the parameters in the gated unit. The purpose is to ensure that the fused entity embedding can achieve optimal performance on the EA task. The specific loss function is shown in Eq. (16):

$$H(y, \widehat{y}) = -[y * log\widehat{y} + (1 - y) * log(1 - \widehat{y})] + \lambda * (w_1 + w_2 + w_3 - 1)^2 \tag{16}$$

where y is the real label, which represents the actual situation of whether the entity or entity pair matches in the entity alignment. Typically, this is a binary value, with 0 indicating that the entity pair does not match and 1 indicating that the entity pair matches; while $\widehat{y}$ is the probability of the entity or entity pair matching predicted by the dataset and pre-aligned seeds, a value close to 1 indicates that the model is highly confident that the entity pair is a match. Close to 0 indicates that the model predicts an entity pair that does not match; $H(y, \widehat{y})$ measures the inconsistency between the model prediction $\widehat{y}$ and the true label y; λ is a regularization parameter that regulates the loss function. Ideally, the sum of w_1 w_2, and w_3 is 1. When the sum of weights deviates from 1, the regularization term $\lambda * (w_1 + w_2 + w_3 - 1)^2$ will penalize the loss function, forcing the learning process to gradually adjust the weights to meet the constraint of w_1+w_2+w_3=1.

3.7. Entity alignment

In the entity alignment stage, we evaluate the similarity between entities by calculating distances. Following the work of Wang et al. (Lan et al., 2018), we also choose to adopt Manhattan distance to measure the distance between entities. For the entity e_i in the TKG_1 and the entity e_j in TKG_2, the distance calculation formula between the entity pair (e_i, e_j) is as shown in Eq. (17):

$$dist(e_i, e_j) = \left|E_{e_i} - E_{e_j}\right| \tag{17}$$

In order to find the aligned entity of entity e_i of TKG_1, we need to calculate its distance to all entities in TKG_2 and select the nearest one as the corresponding entity of e_i in TKG_2.

3.7.1. Training module

To make equivalent entities in the same language close to each other in a unified vector space, we adopt the following margin-based loss function, as shown in Eq. (18):

$$\begin{aligned} L = &\sum_{(e_i, e_j) \in S} ReLU\left(dist(e_i, e_j) - dist(e_i', e_j) + \lambda\right) \\ &+ ReLU\left(dist(e_i, e_j) - dist\left(e_i, e_j'\right) + \lambda\right) \end{aligned} \tag{18}$$

where $ReLU$ is a modified linear unit function, which is used to compare the input value of the function with 0 and retain the larger value; here e_i' and e_j' are negative sample entities that should not be close to e_i and e_j; λ is a Adjust parameters.

3.7.2. Optimization module

In the entity alignment task, the distance measurement between entities is a key link. We introduce a distance threshold θ as the criterion for judging whether two entities are aligned. Specifically, when the distance between any two entities is less than θ, we classify them as an aligned entity pair; conversely, if the distance between them is greater than θ, then the two entities are considered to be not aligned.

Table 2
Datasets.

Datasets		Entities	Relations	Attributes	Temporal	Rel. triples	Attr. triples
DBP-WDT	DBpedia	100,000	321	346	95,070	463,210	380,834
	Wikidata	100,000	216	713	93,071	448,694	789,775
DBP-YGT	DBpedia	100,000	302	334	70,414	428,952	451,646
	YAGO3	100,000	31	23	101,265	502,563	118,376

Given that different data sets have unique characteristics, a single fixed distance threshold θ may not be suitable for all scenarios. In this case, in order to enhance the flexibility of the Tem-DA entity alignment framework, we adopted a dynamic threshold adjustment strategy. In the initial stage of training, an initial threshold θ_{init} is set.

Next, the Tem-DA model will be trained on the known aligned entity set S, and dynamically adjust θ through continuous iterative optimization. In this process, the goal of the model is to find the final threshold θ_{final} that maximizes alignment performance. To achieve this goal, we not only repeatedly evaluate the performance of the model on the training data, but also adjust θ through cyclic testing on the mini-batch validation set.

In order to overcome the limitations in the size of the initial alignment seed entity set S and expand the training data set, the BootEA framework uses a bootstrap iterative training method to significantly improve performance in the entity alignment task. Following this strategy, during each iteration, we select aligned entity pairs (e_i, e_j) and include them into the training data of the next iteration. The algorithm of the entity alignment optimization module is shown in Algorithm 1.

The algorithm of the entity alignment optimization module is divided into the following stages: initialization stage (01). In this stage, the initial distance threshold θ is set to θ_{init}. This value will be used in the first round of iteration to determine whether the entity pairs are similar enough. Iterative training phase (02–14). In this stage, the distance is calculated for each entity pair (e_i, e_j) in the pre-aligned seed entity set S. If it is less than θ, it is added to the aligned entity pair set P. And update the distance threshold θ according to the effect of aligned entities on the set P trained on S. If the alignment entity does not significantly expand the set P, the distance threshold θ is increased. On the contrary, if the expansion is large, the distance threshold θ is decreased. If the model detects convergence on the pre-aligned seed entity set S, update θ_{final} and stop iteration. Update seed pair set stage (15–20). This stage is based on the previously determined final distance threshold θ_{final}. If the calculated distance is less than θ_{final}, the aligned entity pair set P and the pre-aligned seed entity set S are updated.

4. Experiment

4.1. Datasets

We evaluate the performance of Tem-DA using the DWY100 K (Melnychuk et al., 2020) dataset in experiments. DWY100 K consists of two subsets: DBP-WD extracted from DBpedia and Wikidata and DBP-YG extracted from DBpedia and YAGO3, each with 100,000 reference entity alignments and more than 900,000 triples. Then, we extracted the corresponding time information from these data sets and merged them with the source data sets into a unified data format, ultimately forming two experimental data sets: DBP-WDT and DBP-YGT. The specific information of the datasets are listed in Table 2.

4.2. Comparative models

To verify the effectiveness of the model Tem-DA, we selected a series of classic and cutting-edge entity alignment baseline methods for comparative analysis, including: MTransE, JAPE, GCN-Align, AliNet, MuGNN, Tem-EA and DualMatch. Among them, MTransE, JAPE, GCN-Align, AliNet and MuGNN are mainly used as baseline methods in the field of static knowledge graphs, while Tem-EA and DualMatch are used as baseline methods in the field of temporal knowledge graphs. Next, we will give a brief introduction to these methods:

Table 3
Overall performance on DBP-WDT.

Methods	Hits@1	Hits@5	Hits@10	MRR
MtransE (Yang et al., 2017)	33.39	42.91	50.44	0.441
JAPE (Hu et al., 2017)	37.45	49.36	58.57	0.429
GCN-Align (Lan et al., 2018)	41.04	54.84	62.29	0.533
AliNet (Sun et al., 2020)	45.73	53.05	63.38	0.551
MuGNN (Cao et al., 2019)	65.47	73.06	81.36	0.714
Tem-EA (Song et al., 2022)	44.31	57.26	63.00	0.562
DualMatch (Liu et al., 2023)	70.19	75.27	83.69	0.811
Tem-DA	**74.62**	**78.55**	**86.34**	**0.847**

MTransE (Yang et al., 2017) is a multilingual knowledge graph entity alignment embedding model. It converts entities and relationships of different knowledge graphs into vectors, and maps them into a unified vector space through translation, linear transformation and other methods to achieve alignment.

JAPE (Hu et al., 2017) is a method for cross-language knowledge graph entity alignment that utilizes attribute value information to enhance entity alignment. By integrating attribute values, the JAPE model can effectively correspond to similar entities in knowledge graphs in different languages.

GCN-Align (Lan et al., 2018) is a knowledge graph entity alignment model based on GCN. It is able to aggregate neighbor information of an entity node. This model treats attribute triples as relational triples, allowing attribute information to be effectively utilized in the embedding learning process.

AliNet (Sun et al., 2020) improves the expressive ability of embeddings by capturing the heterogeneous neighborhood structure of entities, and combines attention mechanisms and subgraph feature learning to refine entity representations. At the same time, it combines structure and attribute information through feature fusion technology, further enhancing the accuracy of entity alignment.

MuGNN (Cao et al., 2019) captures different dimensional features of entities through multiple channels, and finally fuses these features to obtain a comprehensive entity representation. Each channel may focus on different aspects such as the attributes, structure, or relationships between entities.

Tem-EA (Song et al., 2022) adopts GCN and translation-based models to effectively capture structural and attribute information. In order to improve the accuracy of entity alignment, Tem-EA introduces the concept of nearest neighbor matching.

Table 4
Overall performance on DBP-YGT.

Methods	Hits@1	Hits@5	Hits@10	MRR
MtransE (Yang et al., 2017)	30.18	37.57	48.94	0.376
JAPE (Hu et al., 2017)	32.24	42.19	53.75	0.388
GCN-Align (Lan et al., 2018)	46.28	59.03	69.11	0.594
AliNet (Sun et al., 2020)	44.76	58.18	72.32	0.580
MuGNN (Cao et al., 2019)	69.35	77.04	85.62	0.777
Tem-EA (Song et al., 2022)	44.71	63.22	68.81	0.608
DualMatch (Liu et al., 2023)	72.22	78.93	84.86	0.836
Tem-DA	**76.71**	**80.64**	**90.39**	**0.886**

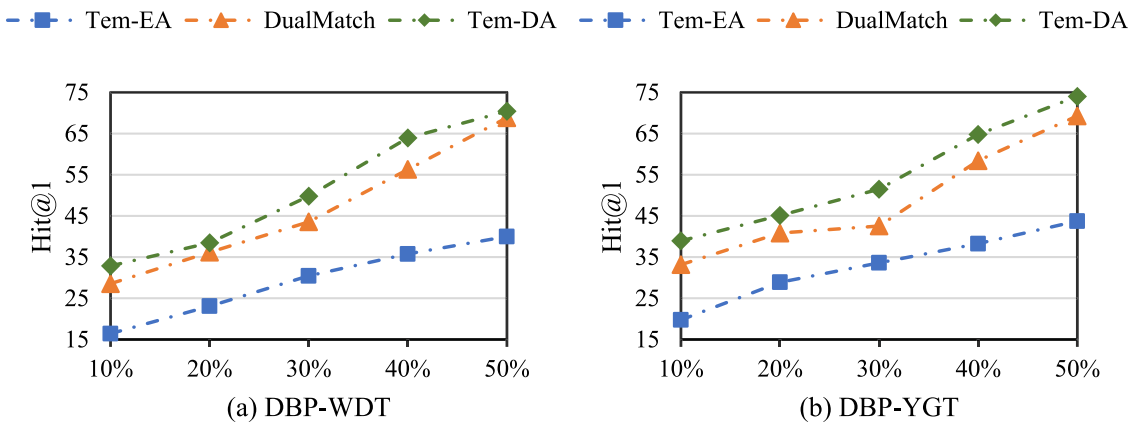

Fig. 5. Entity alignment results of Hits@1 under different seed ratios.

DualMatch (Liu et al., 2023) is a method for temporal knowledge graph entity alignment, which treats the entity alignment task as a weighted graph matching problem. In order to avoid over-reliance on pre-aligned seed entity pairs, DualMatch adopts an unsupervised learning strategy. For time and relationship information, the Dual-Encoder unlabeled encoder is used to encode them separately.

4.3. Experiment results

Tables 3 and Table 4 show the performance comparison of Tem-DA and all baseline models on the temporal knowledge graph datasets DBP-WDT and DBP-YGT, where black bold font indicates the best performance results. Baseline models are divided into two types: static baseline models that do not consider temporal information and baseline models that consider temporal information. The performance of each model is evaluated through the performance indicators Hits@1, Hits@5, Hits@10 and MRR. The higher the score, the better the performance of the model. Experimental results show that Tem-DA outperforms all baseline models on experimental datasets.

Experimental results show that Tem-DA's Hit@1 and MRR indicators on the DBP-WDT data set reached 74.62 % and 0.847 respectively. Compared with the 70.19 % and 0.811 achieved by the latest benchmark model DualMatch, Hit@1 and MRR Increases of approximately 6.3 % and 4.4 % respectively. Similarly, on the DBP-YGT data set, the indicators of Hit@1 and MRR reached 76.71 % and 0.886 respectively. Compared with DualMatch's 72.22 % and 0.836, Hit@1 and MRR achieved approximately 6.2 % and 5.9 % respectively. Performance improvements. These results fully demonstrate that Tem-DA is effective in processing monolingual temporal knowledge graph entity alignment tasks.

Among all benchmark methods, JAPE and MTransE are based on translation models. The reason why JAPE shows better performance than MTransE is that JAPE uses attribute information in the KG to enrich entity embedding. The reason why the performance of GCN-Align based on the GCN model is generally better than that of JAPE and MTransE based on the translation model is that it can effectively capture the neighborhood information around the entity, which is more consistent with the graph structure characteristics of the KG However, GCN-based methods also have shortcomings. The reason is that it uses equal weight when dealing with neighbor nodes, which results in the influence of all neighbor nodes being treated equally during the information aggregation stage. Based on this, AliNet introduces an attention mechanism to further improve the performance of EA, allowing the model to more accurately adjust the representation of entities. Similarly, Tem-DA uses the gating mechanism to optimize the attention mechanism to represent the learning process in the graph attention network. Among the static knowledge graph models, MuGNN performs best because it captures various dimensional information of entities through multi-channel methods, making the feature representation more comprehensive. However, it ignores the possible impact of temporal information on knowledge representation. Tem-EA also uses attribute triples to optimize entity alignment, but the TransE embedding it uses is not as effective as the BERT language model. The reason is that BERT is pre-trained on large amounts of text data, which means it can understand and represent complex attribute information. Based on this, Tem-DA uses the BERT model to process attribute information. In order to avoid over-reliance on aligned seed entity pairs, the DualMatch model adopts an unsupervised learning strategy, but this method does not effectively utilize the differential contributions of entity neighbor nodes.

It is worth noting that the performance of translation-based methods

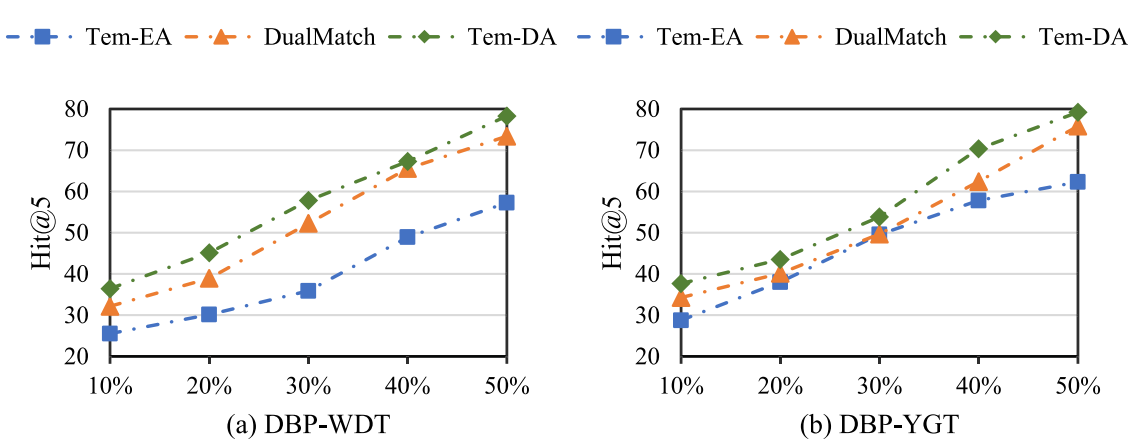

Fig. 6. Entity alignment results of Hits@5 under different seed ratios.

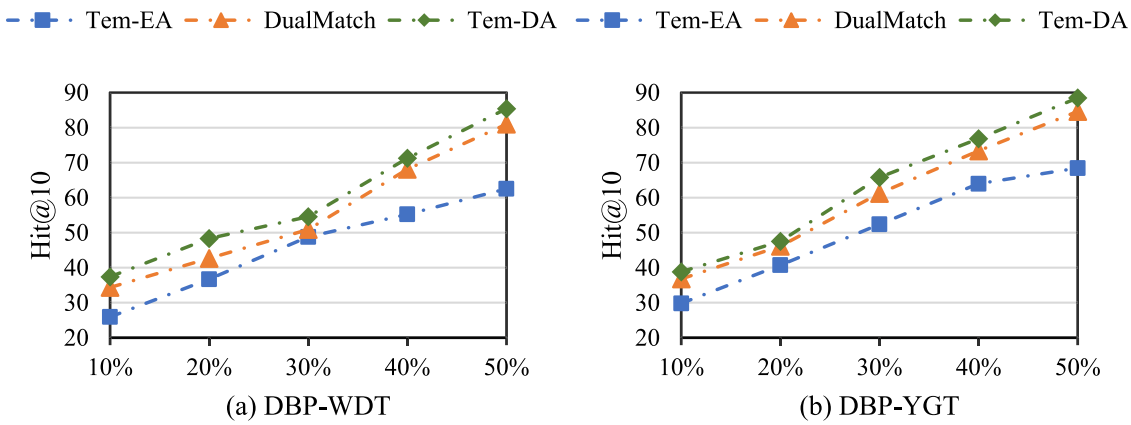

Fig. 7. Entity alignment results of Hits@10 under different seed ratios.

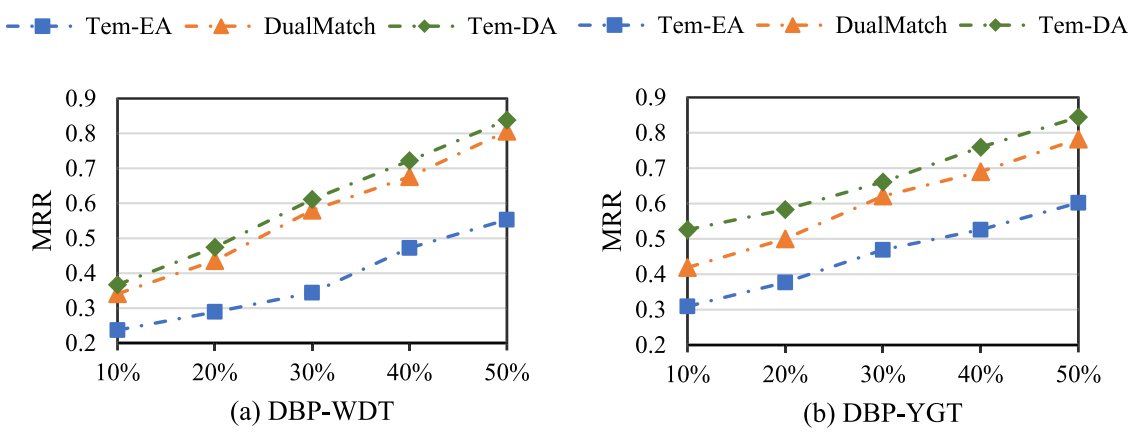

Fig. 8. Entity alignment results of MRR under different seed ratios.

(MTransE, JAPE) on the DBP-WDT data set is better than the DBP-YGT data set. The reason is that the DBP-WDT data set contains relatively rich entity relationship types and attribute information. Translation-based methods use relationship information or attribute information between entities to better achieve entity alignment. The performance of GNN-based methods (GCN-Align, AliNet, MuGNN, Tem-EA, DualMatch and Tem-DA) on the DBP-YGT data set is better than the DBP-WDT data set. This is because there are fewer relationship types but a rich number of relationships in DBP-YGT, and the GNN-based method can show superior performance on the DBP-YGT data set with strong entity connectivity.

During the experiment of entity alignment, different proportions of alignment seeds will have many effects. For example: (1) By observing the performance changes under different alignment seed ratios, you can better understand the impact of parameter adjustment on model performance; (2) By changing the alignment seed ratio, you can test the robustness of the model in different situations. Therefore, this paper analyzes the Hit@k (k = 1,5,10) and MRR performance indicators of Tem-EA, DualMatch and Tem-DA under different seed ratios on two data sets DBP-WDT and DBP-YGT. Impact. The specific experiment set five different seed ratios of 10 %, 20 %, 30 %, 40 % and 50 %, gradually increasing in steps of 10 %. As shown in Fig. 5, Fig. 6, Fig. 7 and Fig. 8, as the seed ratio increases, the performance of both models improves. Compared with Tem-EA and DualMatch, the Tem-DA model shows better results in tests of various seed ratios. Tem-DA performs best when the seed ratio rises to 50 %.

Table 5
Results of ablation experiments on DBP-WDT dataset.

Dataset	DBP-WDT			
Metrics	Hits@1	Hits@5	Hits@10	MRR
Tem-DA(w/o Timing)	56.85	61.22	68.53	0.519
Tem-DA(w/o Attr)	67.18	75.37	79.22	0.706
Tem-DA(w/o Eom)	70.17	75.52	82.23	0.788
Tem-DA	**74.62**	**78.55**	**86.34**	**0.847**

Table 6
Results of ablation experiments on DBP-YGT dataset.

Dataset	DBP-YGT			
Metrics	Hits@1	Hits@5	Hits@10	MRR
Tem-DA(w/o Timing)	60.61	64.82	68.27	0.675
Tem-DA(w/o Attr)	71.09	75.22	81.27	0.720
Tem-DA(w/o Eom)	73.71	78.53	85.62	0.818
Tem-DA	**76.71**	**80.64**	**90.39**	**0.886**

4.4. Ablation study

To better improve the Tem-DA model, three ablation experiments were designed on DBP-WDT and DBP-YGT data sets: Tem-DA (no time information processing, w/o Timing), Tem-DA (no entity alignment optimization, w/o Eom) and Tem-DA (no attribute information processing, w/o Attr). Among them, Tem-DA (w/o Timing) means that there is no time to embed the module, and no time information is processed; Tem-DA (w/o Eom) means that only the entity optimization link

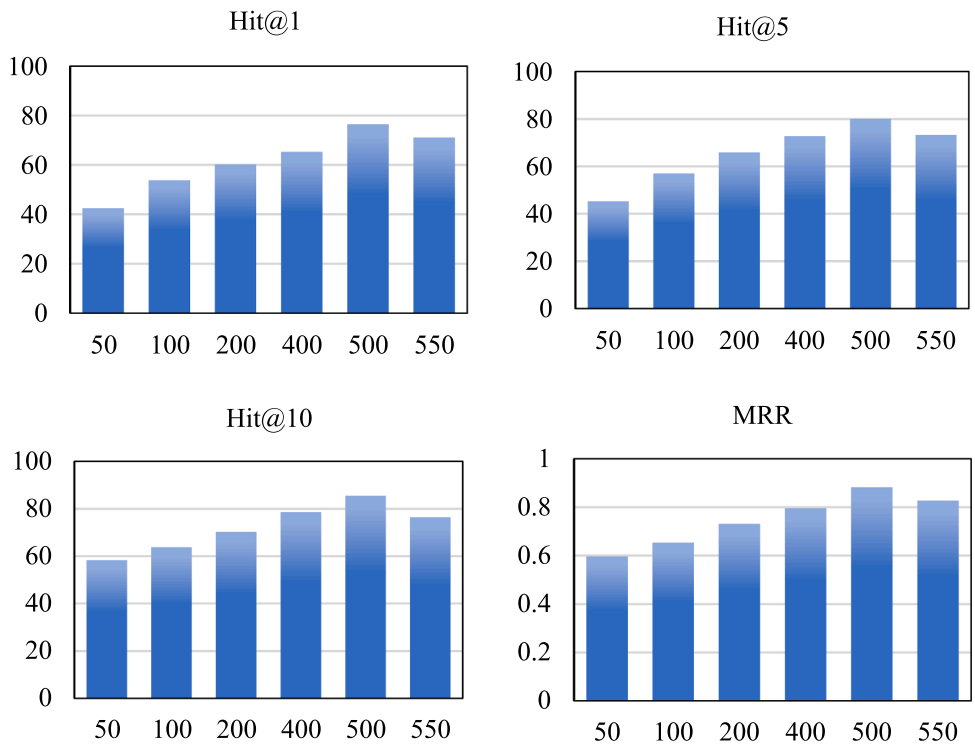

Fig. 9. Performance of Tem-DA with different embedding dimensions.

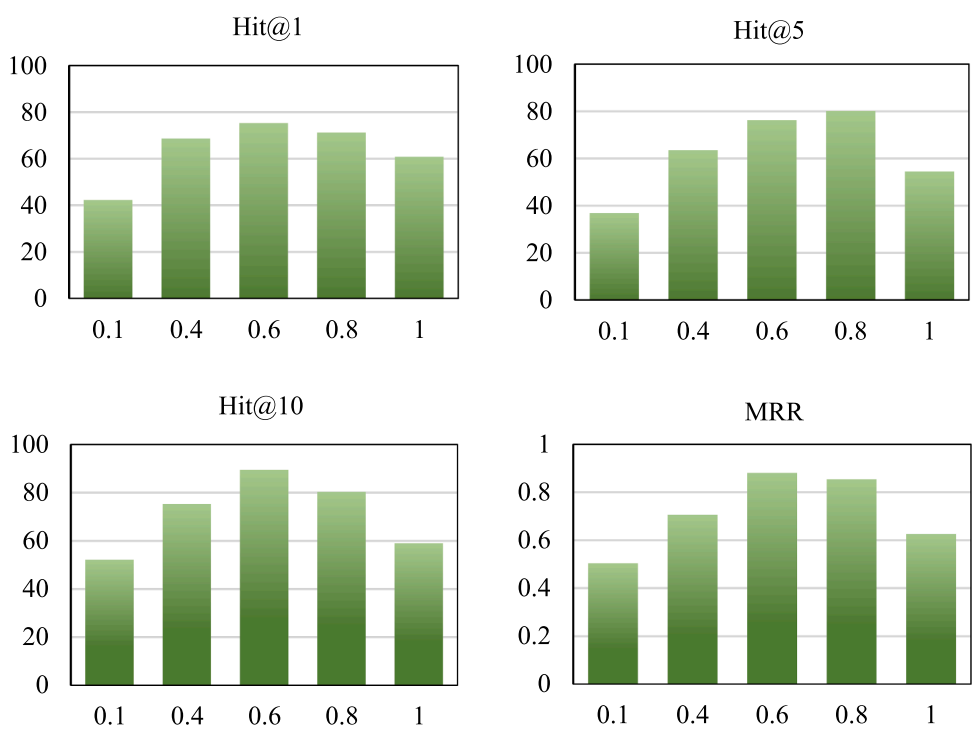

Fig. 10. Performance of Tem-DA with different distance thresholds.

is removed; Tem-DA (w/o Attr) means that attribute information is no longer processed. According to the experimental results in Table 5 and Table 6, compared with the complete Tem-DA model, the above three model variants all show different degrees of performance degradation, where the black bold font in the table indicates the best performance results. From this we can observe that in particular Tem-DA (w/o Timing) is the most affected, followed by TNEA (w/o Attr) and finally Tem-DA (w/o Eom). Through the experimental results, we can clearly see that the modules of attribute information processing, time information embedding and entity alignment optimization are very important to improve the entity alignment performance of TKG, among which time information processing is particularly critical, followed by the use of attribute information and the optimization strategy of entity alignment.

4.5. Parameter analysis

To ensure the maximum performance of the EA model, Tem-DA conducts a hyperparameter search in the following parameter information. Specifically, Tem-DA examined five different learning rates (0.0001, 0.0005, 0.001, 0.005, 0.01); To determine the accuracy of entity matching, the experiment tested five distance thresholds θ (0.1, 0.4, 0.6, 0.8, 1); the regularization coefficient λ of the cross-entropy loss function (0.01, 0.1, 1); in addition, it also includes the representation dimensions of each hidden layer (50, 100, 200, 400, 500, 550) and the number of negative entity pairs (5, 10, 15, 20). By comparing the impact of different parameter configurations on performance, Tem-DA finally selected the parameter configuration as follows: the distance threshold θ is 0.6, the regularization coefficient λ is 0.1 and the learning rate is 0.001. For the data sets DBP-WDT and DBP-YGT, a batch size of 10,000 was set during the experiment, and 10 negative samples were extracted for each pre-aligned entity pair. This study also implemented an early stopping strategy based on the model's performance on the indicator Hits@1 to ensure the optimization of results. Using the control variable method, the performance analysis of Tem-DA with different embedding dimensions and different distance thresholds is shown in Fig. 9 and Fig. 10 respectively.

4.6. Computational complexity analysis

First, the time complexity of the embedding process of attribute names/descriptions is $O\left(d_{bert} \times \left(|V_{tkg1}| + |V_{tkg2}|\right)\right)$, where $|V_{tkg1}|$ and $|V_{tkg2}|$ represent the number of attribute names/descriptions in the two TKGs; d_{bert} is the embedding dimension of the BERT model. Next, the time complexity of graph attention network embedding is $O\left(d_{att} \times \left(|A_{tkg1}| + |A_{tkg2}|\right) + d_{rel} \times \left(|R_{tkg1}| + |R_{tkg2}|\right) + d \times \left(|E_{tkg1}| + |E_{tkg2}|\right)\right)$,$d_{att}$,$d_{rel}$ and d are the dimensions of attribute embedding, relationship embedding and entity structure embedding respectively. Then, temporal information embedding is dominated by multi-layer graph convolutional networks, and the time complexity is approximately $O\left(L \times |E|^2\right)$. Finally, the time complexity of the entity alignment module is $O(T \times |S| + |D|)$, which consists of two parts: (1) T iterative training and threshold updating based on the seed alignment set S, the time complexity is $O(T \times |S|)$, where T is the number of iterations and $|S|$ is the size of the seed alignment set; (2) Traverse the entire data set D to calculate the distance between entities, with a complexity of $O(|D|)$.

4.7. Defects analysis

Although Tem-DA has achieved remarkable results in experimental results on two temporal knowledge graph datasets. However, there are still some areas for improvement and optimization. Tem-DA relies on pre-aligned entity datasets. This means that when the knowledge graph data type changes, the data may need to be re-collected and annotated. In order to reduce the dependence on such annotated data sets, semi-supervised or unsupervised learning strategies may be explored.

5. Conclusion

This paper introduces a new entity alignment method, Tem-DA. Tem-DA first uses GAT combined with the distance detection module to embed the direct neighbors and indirect neighbors of the entity to construct structural features, and combines the gating mechanism during the embedding process to finely adjust and fuse multi-level feature representations. Temporal information is then encoded based on the characteristics of temporal overlap between entities. For entities that lack sufficient temporal information, Tem-DA generates temporal embedding vectors by estimating the temporal characteristics of adjacent information. To represent attribute information, the BERT model is used to construct features. During the entity alignment process, in order to fuse different types of features, the GRU is used to complete the adaptive adjustment of different feature embeddings, providing a more flexible and adaptive feature fusion solution than the traditional linear weight mediation method. In addition, to make up for the insufficient set of pre-aligned entity pairs, Tem-DA introduces a mechanism that combines dynamic distance threshold adjustment with a bootstrap iteration strategy. Experimental results on two monolingual temporal knowledge graph datasets show that Tem-DA achieves remarkable results in the monolingual entity alignment task and exceeds existing baseline models.

CRediT authorship contribution statement

Lin Zhu: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Formal analysis. **Guishun Li:** Writing – original draft, Validation, Methodology, Investigation, Formal analysis. **Luyi Bai:** Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The work was supported by the National Natural Science Foundation of China (61402087), the Natural Science Foundation of Hebei Province (F2022501015), and the Fundamental Research Funds for the Central Universities (2023GFYD003).

Data availability

Data will be made available on request.

References

Bai, L., Li, N., Li, G., Zhang, Z., & Zhu, L. (2024). Embedding-based entity alignment of cross-lingual temporal knowledge graphs. *Neural Networks, 172*, Article 106143.

Biega, J., Mahdisoltani, F., & Suchanek, F. M. (2013). Yago3: A knowledge base from multilingual wikipedias. In *Proceedings of the 2nd Conference on Innovative Data Systems Research* (pp. 22–41).

Cao, Y., Liu, Z., Bai, L., Li, J., & Chua, T. S. (2019). Multi-channel graph neural network for entity alignment. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 1452–1461).

Chen, J., Yang, L., Wang, Z., & Gong, M. (2024b). Higher-order GNN with Local Inflation for entity alignment. *Knowledge-Based Systems*, Article 111634.

Chen, J., Yang, L., Wang, Z., & Shang, F. (2024a). Relation mapping based on higher-order graph convolutional network for entity alignment. *Engineering Applications of Artificial Intelligence, 133*, Article 108009.

Chen, M., Tian, Y., Chang, K. W., Skiena, S., & Zaniolo, C. (2018). Co-training embeddings of knowledge graphs and entity descriptions for cross-lingual entity alignment. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence* (pp. 3998–4004).

Dai, J., Zhang, W., Chen, M., Hu, W., Wang, C., Sun, Z., et al. (2020). Knowledge graph alignment network with gated multi-hop neighborhood aggregation. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence* (pp. 222–229).

Ding, Z., Ma, Y., He, B., Wu, J., Han, Z., & Tresp, V (2023). A simple but powerful graph encoder for temporal knowledge graph completion. In *Intelligent Systems Conference* (pp. 729–747).

Dumančić, S., García-Durán, A., & Niepert, M. (2018). Learning sequence encoders for temporal knowledge graph completion. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 4816–4821).

Feng, J., Wang, Z., Zhang, J., & Chen, Z. (2014). Knowledge graph embedding by translating on hyperplanes. In *Proceedings of the 28th AAAI Conference on Artificial Intelligence* (pp. 1112–1119).

Gai, K., Asamoah, C., Tao, L., & Jiang, N. (2016). Powering filtration process of cyber security ecosystem using knowledge graph. In *Proceedings of the IEEE 3rd International Conference on Cyber Security and Cloud Computing* (pp. 240–246).

Garcia-Duran, A., Weston, J., Bordes, A., Usunier, N., & Yakhnenko, O. (2013). Translating embeddings for modeling multi-relational data. In *Proceedings of the 26th International Conference on Neural Information Processing Systems* (pp. 2787–2795).

Goel, R., Kazemi, S. M., Brubaker, M., & Poupart, P. (2020). Diachronic embedding for temporal knowledge graph completion. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence* (pp. 3988–3995).

Han, Zhen, et al. "Dyernie: Dynamic evolution of riemannian manifold embeddings for temporal knowledge graph completion." arXiv preprint arXiv:2011.03984 (2020c).

Han, Zhen, et al. (2020a). Explainable subgraph reasoning for forecasting on temporal knowledge graphs. In *International conference on learning representations*.

Han, Zhen, et al. "Graph hawkes neural network for forecasting on temporal knowledge graphs." arXiv preprint arXiv:2003.13432 (2020b).

Han, Zhen, et al. (2021a). Time-dependent entity embedding is not all you need: A re-evaluation of temporal knowledge graph completion models under a unified framework. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing* (pp. 8104–8118).

Han, Zhen, et al. (2021b). Learning neural ordinary equations for forecasting future links on temporal knowledge graphs. In *Proceedings of the 2021 conference on empirical methods in natural language processing* (pp. 8352–8364).

He, X., Hu, Z., Cao, Y., Wang, X., & Chua, T. S. (2019). Unifying knowledge graph learning and recommendation: Towards a better understanding of user preferences. In *Proceedings of the 19th World Wide Web Conference* (pp. 151–161).

He, X., Yih, S. W., Chang, M. W., & Gao, J. (2015). Semantic parsing via staged query graph generation: Question answering with knowledge base. In *Proceedings of the Joint Conference of the 53rd Annual Meeting of the ACL* (pp. 1321–1331).

Hu, W., Sun, Z., & Li, C. (2017). Cross-lingual entity alignment via joint attribute-preserving embedding. In *Proceedings of the 16th International Semantic Web Conference* (pp. 628–644).

Ji, G., He, S., Xu, L., Liu, K., & Zhao, J. (2015). Knowledge graph embedding via dynamic mapping matrix. In *Proceedings of the 53rd annual meeting of the association for computational linguistics and the 7th international joint conference on natural language processing* (pp. 687–696).

Kazemi, S. M., & Poole, D. (2018). Simple embedding for link prediction in knowledge graphs. *Advances in Neural Information Processing Systems*, 31.

Krötzsch, M., Mendez, J., Erxleben, F., Günther, M., & Vrandečić, D. (2014). Introducing wikidata to the linked data web. In *Proceedings of the 13th International SemanticWeb Conference* (pp. 50–65).

Lan, X., Wang, Z., Lv, Q., & Zhang, Y. (2018). Cross-lingual knowledge graph alignment via graph convolutional networks. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 349–357).

Lautenschlager, Steve Shellman Jennifer, & Ward, Michael (2015). Icews event aggregations. *Journal of Harvard Dataverse*, 3.

Li, C., Cao, Y., Hou, L., Shi, J., Li, J., & Chua, T. (2019). Semi-supervised entity alignment via joint knowledge embedding model and cross-graph model. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 2723–2732).

Liu, X., Wu, J., Li, T., Chen, L., & Gao, Y. (2023). Unsupervised entity alignment for temporal knowledge graphs. In *Proceedings of the 32nd ACM Web Conference* (pp. 2528–2538).

Liu, Yushan, et al. (2022). Tlogic: Temporal logical rules for explainable link forecasting on temporal knowledge graphs. *Proceedings of the AAAI Conference on Artificial Intelligence, 36*(4), 4120–4127.

Liu, Z., Zhu, H., Xie, R., & Sun, M. (2017). Iterative entity alignment via joint knowledge embeddings. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence* (pp. 4258–4264).

Ma, Y., Tresp, V., & Daxberger, E. A. (2019). Embedding models for episodic knowledge graphs. *Journal of Web Semantics, 59*, Article 100490.

Mao, X., Wang, W., Xu, H., & Wu, Y. (2020). MRAEA: An efficient and robust entity alignment approach for cross-lingual knowledge graph. In *Proceedings of the 13th International Conference on Web Search and Data Mining* (pp. 420–428).

Melnychuk, V., Tresp, V., Faerman, E., Berrendorf, M., & Seidl, T. (2020). Knowledge graph entity alignment with graph convolutional networks: Lessons learned. In *Proceedings of the 42nd European Conference on IR Research* (pp. 3–11).

Song, X., Bai, L., & Zhang, H. (2022). Temporal knowledge graph entity alignment via representation learning. In *Proceedings of the 27th Database Systems for Advanced Applications* (pp. 391–406).

Su, F., Xu, C., & Lehmann, J. (2021). Time-aware graph neural network for entity alignment between temporal knowledge graphs. In *Proceedings of the 2021 Empirical Methods in Natural Language Processing* (pp. 8999–9010).

Sun, M., Liu, Y., Lin, Y., Liu, Z., & Zhu, X. (2015). Learning entity and relation embeddings for knowledge graph completion. In *Proceedings of the 29th AAAI Conference on Artificial Intelligence* (p. 1).

Sun, Z., Wang, C., Chen, M., Hu, W., Dai, J., Zhang, W., et al. (2020). Knowledge graph alignment network with gated multi-hop neighborhood aggregation. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence* (pp. 222–229).

Tresp, V., Ma, Y., Baier, S., & Yang, Y. (2017). Embedding learning for declarative memories. In *The Semantic Web: 14th International Conference, ESWC 2017, Portorož, Slovenia, May 28–June 1, 2017, Proceedings, Part I 14* (pp. 202–216). Springer International Publishing, 2017.

Trisedya, B. D., Qi, J., & Zhang, R. (2019). Entity alignment between knowledge graphs using attribute embeddings. In *Proceedings of the 33rd AAAI Conference on Artificial Intelligence* (pp. 297–304).

Trivedi, R., Dai, H., Wang, Y., & Song, L. (2017). Know-evolve: Deep temporal reasoning for dynamic knowledge graphs. In *Proceedings of the 34th International Conference on Machine Learning* (pp. 3462–3471).

Wang, W., Wu, Y., Mao, X., & Lan, M. (2021). From alignment to assignment: Frustratingly simple unsupervised entity alignment. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing* (pp. 2843–2853).

Wang, Z., You, X., & Lv, X. (2024). A relation enhanced model for temporal knowledge graph alignment. *Journal of Supercomputing, 80*(5), 5733–5755.

Wu, Y., Liu, X., Wang, Z., Feng, Y., & Zhao, D. (2019a). Jointly learning entity and relation representations for entity alignment. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing* (pp. 240–249).

Wu, Y., Liu, X., Wang, Z., Feng, Y., & Zhao, D. (2020). Neighborhood matching network for entity alignment. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*.

Wu, Y., Liu, X., Yan, R., Wang, Z., Feng, Y., & Zhao, D. (2019b). Relation-aware entity alignment for heterogeneous knowledge graphs. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence* (pp. 5278–5284).

Xie, R., Wu, W., Ma, M., Yuan, H., Zhu, J., Wang, Z., et al. (2022). An effective and efficient entity alignment decoding algorithm via third-order tensor isomorphism. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics* (pp. 5888–5898).

Xiong, B., Xu, C., Su, F., & Lehmann, J. (2022). Time-aware entity alignment using temporal relational attention. In *Proceedings of the 21th World Wide Web Conference* (pp. 788–797).

C. Xu, M. Nayyeri, H.S. Yazdi and J. Lehmann, Temporal knowledge graph embedding model based on additive time series decomposition, CoRR abs/1911.07893 (2019) Han, Zhen, et al. "Dyernie: Dynamic evolution of riemannian manifold embeddings for temporal knowledge graph completion." arXiv preprint arXiv:2011.03984 (2020).

Yang, B., Yih, W., He, X., Gao, J., & Deng, L. (2015). Embedding entities and relations for learning and inference in knowledge bases. In *Proceedings of the 3rd International Conference on Learning Representations*.

Yang, M., Chen, M., Tian, Y., & Zaniolo, C. (2017). Multilingual knowledge graph embeddings for cross-lingual knowledge alignment. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence* (pp. 1511–1517).

Yu, M., Feng, Y., Song, Y., Wang, Z., Xu, K., Wang, L., et al. (2019). Cross-lingual knowledge graph alignment via graph matching neural network. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 3156–3161).

Zhang, Q., Sun, Z., Hu, W., & Qu, Y. (2018). Bootstrapping entity alignment with knowledge graph embedding. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence* (pp. 4396–4402).